A 209-state candidate for the consistency of Peano arithmetic

Making three contextual decoder reductions compatible

Research prepared for Vladimir Reshetnikov

29 October 2026

Abstract

We compare the three 211-state proposals in the specified ProveIt incoming directory and construct a binary, single-tape machine with **209 working states and 418 transitions**, including its blank-tape initializer. A separately verified 210-state intermediate is also supplied. The three input machines have the same numerical proof-search program, but their rooted labeled transition graphs are pairwise nonisomorphic. Their savings remove different native states and cannot be stacked by copying patches without preserving their entry conditions.

The new construction first combines the Research and Rotated savings by eight closed source reassociations. It then changes a 720-primitive region across a decoder shared with the runtime, preserving the specific branches used by that runtime entry. An eight-factor reassociation and three further rotations make the Contextual saving compatible with the other two. Native states 181, 218, and 180 are all absent from the final table.

Fresh symbolic derivations establish all 5,084 guarded primitive contracts for the final table, with arbitrary natural registers, arbitrary extra header gap, arbitrary nonempty finite spare lists, and complete-tape endpoints. Every literal row is covered. Independent finite checkers verify the source transformation, entry repair, local folds, and exact state counts; the complete 48,140-step blank initialization is unchanged. Composing these new arguments with the retained numerical-calculus and unary-theory calibration yields the ordinary mathematical, PA-formalizable endpoint $\mathrm{PA} \vdash \neg\,\mathrm{Halt}(U_{209}, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathrm{PA})$. This package supplies mathematical proofs and executable certificates, not a newly exported Lean, Rocq, MM0, or PA proof object for that endpoint.

Primary artifact: `machine/u209.tm`. Working states $0, \ldots, 208$; separate halt label 209; start state and head coordinate 0; all-zero initial tape; one two-way infinite tape; alphabet $\{0, 1\}$; only left and right moves. All $209 \cdot 2 = 418$ working-state/symbol pairs are defined.
Secondary artifact: `machine/u210.tm`. The same conventions, with working states $0, \ldots, 209$ and halt label 210.

1 Result, scope, and exact identities

The requested threshold is surpassed by two states. Relative to the common 212-state parent, all three distinct published one-state savings now coexist: the Research deletion of 181, the Rotated deletion of 218, and a contextual deletion of 180 using a newly arranged keeper 216. No extra alphabet symbol, prewritten input, additional tape, or omitted initializer is used.

A literal row `q b w D r` reads b in state q, writes w, moves one cell in direction D, and enters r. Entering the separate halt label ends the computation after the write and move. Native tables keep sparse inherited labels and halt label 254; these labels are names, not counts. The dense tables are

Property	U_{210}	U_{209}
Working states	210	**209**
Transition rows	420	**418**
Source primitive occurrences	3,909	3,909
Distinct syntactic subtrees	197	197
Maximum source-address depth	41	41
Maximum consecutive zeros in an address	6	6
Nonhalt source successor edges	5,083	5,083
Guarded primitive cases, including halt	5,084	5,084

Table 1: The savings come from contextual control-state sharing, with the same source program and the same number of distinct syntactic subtrees.

exact checked renamings. Every working state is reachable in the transition graph. This does not assert that every state is visited on the particular blank run.

The primary dense-table SHA-256 is

`bda91bd40880f96edef104246d22797fe281355cf03610ab7e93e7923be8aa16`.

Its native-table and source-map hashes are, respectively,

`4564c4876f9b18725bfb3505ec1f50d1ebce572502f2c6381ad673ecfdc3c3e1`,
`912724f29b0de28b3326c710fdf1f3863ebd07058c6db1a9b4bbaffb3ae5e3f3`.

The complete 209-state table appears in Appendix A. Both state bijections, both literal tables, and all intermediate tables needed by the local proofs are included as machine-readable files.

Theorem 1.1 (New halting-preserving construction)**.** *Let A be the exact Research U211 machine retained in this package. The source identities, final-table primitive simulations, and common blank initialization described below give PA-formalizable proofs of*

$$\mathrm{Halt}(U_{209}, \mathrm{blank}) \iff \mathrm{Halt}(U_{210}, \mathrm{blank}) \iff \mathrm{Halt}(A, \mathrm{blank}).$$

Theorem 1.2 (Consistency endpoint with the retained calibration)**.** *Composing Theorem 1.1 with the retained numerical proof-search and Theory U calibration gives, in ordinary mathematics,*

$$\mathrm{PA} \vdash \neg\,\mathrm{Halt}(U_{209}, \mathrm{blank}) \iff \mathrm{Con}(U) \iff \mathrm{Con}(\mathrm{PA}).$$

Here *retained calibration* names mathematical dependencies rather than a new object-theory assumption. In particular the biconditional does not assume Con(PA). Actual unconditional nonhalting is not established. The distinction between an ordinary PA-formalizable argument and an exported, kernel-checked PA endpoint is maintained throughout this report.

2 Comparison of the three supplied proposals

2.1 Snapshot and literal comparison

The comparison is pinned to ProveIt commit

`da4ab9dd68ccfc47ac852118601eaf1fdae53795`.

Its `docs/incoming/` directory contained the three U211 archives and `pa_unary_power_cover.zip` used here [1]. All four original archives are retained unchanged in `inputs/`. Their SHA-256 values, original paths, and sizes are recorded in `provenance.json`. The table/source snapshots used by the new construction are checked directly against the corresponding ZIP members.

Proposal	Rotations	Deleted label	Principal mechanism
Contextual	6	180	Predecessor-tag fold into 163 through helper 211; also repairs the backward entry 171/0 into state 7.
Research	9	181	Predecessor-tag fold into 220 through helper 129; retains the earlier helper 211.
Rotated	11	218	Makes every entry to 218 write a zero immediately left of the head, then replaces 218 by the existing state 157.

Table 2: Distinct 211-state constructions from the same U212 parent.

All three contain the same ordered 3,909 primitive operations and operands. Both source-control resolvers produce the same complete ordinal control-flow graph. However, their native tables differ, and canonical breadth-first numbering from state 0 shows that their rooted transition graphs are pairwise nonisomorphic when read symbols, write symbols, and directions are preserved. These are different machines, not three filenames for one table.

Compared pair	Different common native rows	Different source addresses
Contextual / Research	28	1,275
Contextual / Rotated	31	3,096
Research / Rotated	29	3,096

Table 3: The row counts exclude the two outgoing rows of each differently deleted label. Full inventories are in `comparison/census.json`.

The common parent has native SHA-256

`386ce16259d4c97e4af31b3ba494feaff11a0a1a066e0235dddc154b4cea38c4`.

The three U211 native identities are listed below in abbreviated form; the full hashes are retained in the comparison files.

Contextual: `ac75e2fa934e89b3...`
Research: `7edae566b4c2e5c3...`
Rotated: `72234980bd604524....`

2.2 Verification quality and the selected parent

Each original package was copied to an isolated working directory and its finite replay command was rerun successfully: Contextual's default reproduction, Research's `-quick` reproduction, and Rotated's default verifier. Those fresh receipts and logs are included. The original full symbolic runtime suites were not rerun for all three inputs; their packaged full-runtime receipts remain prior evidence. In contrast, the complete final-table runtime derivations for both new candidates were freshly regenerated.

The Research table is the direct parent of the new work because its tagged fold can be preserved while the 218 predecessor condition is rearranged. It also gives a particularly transparent 210-state

intermediate without the 171-entry repair. The Contextual package contributes an independently useful retagging and repair argument; Rotated contributes the exact 218-to-157 two-cell identity. None of these merits implies that its entire patch can be copied into another source layout without rechecking the entry conditions.

The common consistency calibration is not a reason to treat all three as already covered by K215's separate formal endpoint. The linked K215/MM0 project proves a statement about a different literal table and ZFC [3]. Its result motivates the architecture but does not certify the newly supplied PA machines.

3 Theory U and the arithmetic program actually preserved

3.1 The unary schema

Theory U uses classical, equality-free first-order logic on nonempty domains, with one binary relation $x \in a$. Put

$$\mathrm{Sub}(x, a) \;:\Longleftrightarrow\; \forall z\,(z \in x \to z \in a).$$

For every formula $\varphi(x)$ with no other free variable, its sole schema is

$$\forall a\,\exists b\,\left[\forall x\,(x \in b \to \varphi(x)) \;\wedge\; \forall x\,(\mathrm{Sub}(x, a) \to (\varphi(x) \to x \in b))\right]. \tag{U}$$

Variables are freshened before insertion. The source a is not a permitted free matrix parameter. This is a restriction on free roles, not a restriction to formulas containing only one variable symbol.

The schema supplies a weak cover: all members of b satisfy the matrix, and every subset of a satisfying it belongs to b. It does not require every member of b to be a subset of a, and it does not postulate ambient extensionality. Replacing the matrix by $\mathrm{Sub}(x, a) \wedge \varphi(x)$ would introduce the forbidden free source a; that apparent shortcut is not available in U.

3.2 Why the retained U/PA proof is substantive

The supplied unary manuscript [2] contains a detailed ordinary interpretation argument. Its upper direction interprets membership as binary bit membership in PA. Bit inclusion implies $x \leq a$; for each fixed arithmetical matrix, PA's induction constructs the finite characteristic code on the interval through a. This is formula-specific finite comprehension, not a uniform truth predicate, and works inside arbitrary PA models.

For the lower direction, equality is interpreted by coextensionality. The numerical representatives explicitly include saturation and suitable first-order order/endpoint conditions. Local predecessor facts establish induction before total successor existence is used. Comparison and least failing numerical tuples then allow parameters to be bound inside a legal unary matrix. Bounded numerical comprehension follows, with its support and invariance premises stated explicitly. This provides successors, numerical powers, pair and graph codes, and canonical finite histories for addition and multiplication. The order of these steps avoids a circular appeal to total successors.

The manuscript also describes uniform syntactic proof templates: freshening, substitution, relativization, replacement of terms by functional graphs, and concatenation of finite derivations. These are primitive recursive syntax operations, with correctness justified by induction in PA. An independent review here found no concrete defect in that dependency chain. The review does not claim to have implemented a complete arbitrary-proof translator or checked an exported endpoint in a proof assistant.

3.3 The machine-facing calculus is a different presentation

The preserved source program searches the inherited numerical calculus $\mathcal{D}_U$. It uses truth code 0, falsity code 1, native Cantor-coded constructors, seven logical rows, a dominated propositional row, modus ponens, generalization, and a gated cover/vacuity bundle. Its machine-facing theory allows additional matrix parameters and uses global irreflexivity. It is calibrated by finite-refutation translations to the unary U presentation, rather than having literally the same axiom strings.

This distinction is essential. The separate positive dyadic emitter in the unary-theory ZIP is not the program implemented by these U211, U210, or U209 tables. Its operation counts cannot be used as a runtime or state-count certificate for them.

The retained role-emitter proof handles small-code aliases, the truth/falsity codes, the gate, and the possible coincidence of output and source labels. In the coincident-label case, irreflexivity changes the relevant guard to emptiness; an empty source supplies the required cover. Padding and fresh labels supply completeness for arbitrary desired instances. The new reductions leave this numerical calculus and every ordered source primitive unchanged.

4 Source equivalence and contextual decoding

The source is a binary tree whose leaves carry CI (increment), CD (guarded decrement), CB (forward branch), CC (backward branch), or CH (halt). There are respectively 1,558, 1,175, 80, 1,095, and one such occurrences. The leaf address encodes control context, so retaining the printed operation sequence alone is insufficient.

For ordinal i, let $G(i)$ record the primitive, its operand, and its ordered successor ordinals. A decrement has zero and positive successors; halt has none. The first resolver computes successors by binary increment and suffix cutting, followed by zero-padded leaf decoding. The second resolver uses the ordinal intervals of enclosing subtrees. Both check all 5,083 nonhalt successor edges. The arithmetic resolver additionally rejects stray one bits after the decoded target leaf.

A factor is closed and nonhalting when its ancestor-relative branches are resolved within the factor and it contains no halt. Closure is checked in two ways: recursive boundary signatures and independent leafwise counts of the relevant enclosing zero/one edges. Association of closed sequential factors preserves the primitive word and resolved ordinal control graph.

Lemma 4.1 (Source conjugacy)**.** *If two complete source trees have the same ordered primitive word and the same $G(i)$ for every ordinal, then from a common ordinal and numerical register store they have identical stores and ordinals at every source step. They halt after the same number of source steps.*

Proof. Induct on the source-step count. Increment changes the same register in both programs. A decrement tests the same value, so both programs choose the same branch and make the same register change. The two branch operations choose the checked common successor; halt occurs in the same ordinal. No bound on the register values is used. □

A pure physical decoder has rows $(0, R, a)$ and $(1, R, b)$. A tagged physical decoder may implement different logical source subtrees depending on a bit left by its incoming transition. Therefore a source rewrite must preserve the relevant entry context. This is the key to the new result: one may alter a branch of a shared decoder that a runtime call cannot execute, while preserving all branches used by that runtime call.

The source theorem is separate from the final literal-machine theorem. Some construction intermediates have seven consecutive zero address bits, which collide with the inherited origin detector. Their source control graphs are still equivalent, but those intermediate tables are not claimed to implement a complete correct runtime. The final sources have maximum zero run six and pass the complete runtime verification directly.

5 First construction: Research U211 to U210

5.1 Move the continuation while preserving the Research selector

Starting with the Research table, apply these eight left associations:

$$\begin{array}{cccc} (162,203,0), & (203,223,0), & (223,188,0), & (188,177,0), \\ (177,89,0), & (89,174,0), & (174,217,0), & (217,163,0). \end{array}$$

Here $(p,q,0)$ changes $((A,B),C)$ at decoder p, whose zero child is q, into $(A,(B,C))$. The builder verifies pure decoder rows, the complete unique incoming edge to the inner state, and closure of each of the three source factors. The independent checker recovers the affected occurrences by literal source-prefix execution and checks both complete control graphs after every source change.

These associations carry continuation 116 to the right of state 163. The Research selector rows at 220 and 129 remain intact. The old 181 state remains deleted. The final remaining entrances to 218 are exactly

$$158/0 : 0\mathrm{R}218, \qquad 163/0 : 0\mathrm{R}218, \qquad 211/0 : 0\mathrm{R}218. \tag{1}$$

Consequently each visit to 218 from state 0 has a zero immediately left of the head. This is a direct consequence of the preceding write/move, valid for every initial tape, rather than a property guessed from a blank trace.

5.2 The 218-to-157 local identity

The old rows are $218/0 = 0\mathrm{R}90$ and $218/1 = 1\mathrm{R}86$. Redirect all targets of 218 to 157 and delete 218. With the original head at coordinate zero, the existing table executes

current bit	replacement state/head path
0	$157@0 \to 67@{-1} \to 151@0 \to 90@1$
1	$157@0 \to 148@{-1} \to 186@0 \to 86@1.$

(2)

Only cells $-1, 0$ are read. The predecessor is zero by (1); each write preserves its bit, and the exterior is untouched.

Lemma 5.1 (U210 final local fold)**.** *From every common initial tape in state 0, the 211-state prefold table and the delivered U210 halt equivalently. Each old 218 step becomes three new steps; every other old step becomes one new step.*

Proof. Map 218 to 157 and fix all other labels. For ordinary old rows, write and move are unchanged and the target is mapped by this projection. For old state 218, use the exhaustive two cases (2) and the incoming-edge invariant (1). Every replacement block has positive length and no internal halt. Induction on old steps gives the simulation, including equality of the whole tape at its block boundaries. Conversely, if the new machine halts after t steps, simulating t old steps would pass that

point unless the old machine had already halted; positivity and the absence of internal halt exclude a spurious new halt. □

For any halting old run, or aligned finite run prefixes, this final fold gives $T_{\text{new}} = T_{\text{old}} + 2k$, where k counts old visits to 218, and $T_{\text{old}} \le T_{\text{new}} \le 3T_{\text{old}}$. This clock bound is not asserted across the preceding address changes.

The exact U210 dense hash is

`9020e082ef6995d13cd571c325f4d83bf615f4d0a82e77c580353c1127a0d596`.

Its full final-table runtime and blank startup were checked independently of the state-count calculation.

6 Second construction: a compatible U209

6.1 The shared helper that prevents a direct patch transplant

In U210, state 129 is entered both by a right-moving source decoder and by the backward call $220/0 = 0\text{L}129$. Its two incoming rows are $215/1 = 1\text{R}129$ and $220/0 = 0\text{L}129$. Treating it as an ordinary singly entered source decoder would lose one of these uses.

The runtime call begins after 220 has just read and preserved zero. If the predecessor bit is t, its path is

$$220@0 \to 129@{-1} \to \begin{cases} 158@0, & t = 0, \\ 204@0, & t = 1, \end{cases} \to \begin{cases} 157@1, & t = 0, \\ 147@1, & t = 1. \end{cases} \tag{3}$$

Thus the call uses 129's two rows, but only the zero rows of 158 and 204. The row 204/1 is available for a source-layout change without altering this runtime block. This selective preservation is broader than requiring the entire shared decoder to be frozen or to have a single entrance.

6.2 A closed eight-factor reassociation

At the U210 source prefix

`0011001110101011111111`

state 174 represents primitive ordinals $[3016, 3736)$, a region of length 720. Write its eight closed factors in their unchanged order as follows.

Factor	Physical entry	Length	Old suffix	New suffix
A	98	26	00	00
B	158	134	010	010
C	147	52	0110	0110
D	146	108	01110	0111
E	220, zero tag	136	011110	10
F	90	27	011111	1100
G	211	103	10	1101
H	163	134	11	111

Table 4: The zero tag in factor E is part of its physical decoding context.

Replace the source shape by

$$((A,(B,(C,(D,(E,F))))),(G,H)) \longmapsto ((A,(B,(C,D))),(E,((F,G),H))).$$

Every factor is closed and nonhalting. The word and both complete ordinal control graphs are preserved. Physically, this is implemented by seven row changes:

row	before	after
204/1	1R214	1R146
214/0	0R146	0R216
214/1	1R216	1R163
216/0	0R220	0R90
216/1	1R90	1R211
217/0	0R211	0R220
217/1	1R163	1R214

(4)

No state is added. State 216 now decodes the pair $(90, 211)$.

The zero-tag entrance to 220 moves from 216/0 to 217/0; it still writes zero and moves right. Its one-tag entrance 185/1 is unchanged. All rows used by (3) are unchanged. In particular 204/0 is preserved while 204/1 changes. The source entry and backward runtime entry are therefore justified separately. Exhaustive final-prefix and primitive-runtime checks additionally validate the actual literal table.

6.3 Retag state 180 and repair the arithmetic entrance

Apply the three right associations

$$(206, 199, 1), \qquad (205, 206, 1), \qquad (7, 205, 1), \tag{5}$$

where side 1 changes $(A,(B,C))$ to $((A,B),C)$. The inner decoder has its specified unique incoming edge in each case, and the three factors are closed. Both complete ordinal control graphs remain unchanged.

State 7 also has the backward arithmetic entrance $171/0 = 0\mathrm{L}7$. Repair this one row to

$$171/0 = 0\mathrm{L}205. \tag{6}$$

For a current zero at coordinate zero, split on the predecessor:

left bit	old path	repaired path
0	$171@0 \to 7@{-1} \to 86@0$	$171@0 \to 205@{-1} \to 86@0$
1	$171@0 \to 7@{-1} \to 205@0 \to 164@1$	$171@0 \to 205@{-1} \to 206@0 \to 164@1$.

(7)

Both paths preserve the two cells and read no exterior cell. This establishes the entry repair on every such configuration.

The first two stages of (5) have maximum zero run seven; their intermediate tables are not asserted to be complete runtimes. The final stage restores maximum zero run six. The independent receipt records any failed intermediate prefix check as `NOT_A_RUNTIME_INTERMEDIATE`; it checks every prefix of the final table successfully.

6.4 Delete 180 using a five-step contextual path

In the 210-state prefold table R,

q	$\delta(q,0)$	$\delta(q,1)$
216	0R90	1R211
180	0R152	1R211.

Its complete incoming inventories are

$$\mathrm{In}(216) = \{214/0 : 0\mathrm{R}216\}, \qquad \mathrm{In}(180) = \{132/1 : 1\mathrm{R}180,\ 199/1 : 1\mathrm{R}180\}. \tag{8}$$

Neither state is initial. Therefore the old state is encoded by the bit immediately left of the head: zero for 216 and one for 180.

Let $\rho(180) = 216$ and $\rho(q) = q$ otherwise. Redirect both incoming targets of 180 to 216, delete its two rows, and replace

$$216/0 \quad \text{by} \quad 0\mathrm{L}211. \tag{9}$$

Keep $216/1 = 1\mathrm{R}211$. The resulting machine C has 209 states.

Old state	Left bit	Read	New steps	New state/head path
216	0	0	5	$216@0, 211@{-1}, 157@0, 67@{-1}, 151@0, 90@1$
180	1	0	3	$216@0, 211@{-1}, 184@0, 152@1$
216	0	1	1	$216@0, 211@1$
180	1	1	1	$216@0, 211@1$

Table 5: Exhaustive whole-tape identities for the final fold. Each old block has one step. Only the predecessor and current cell are read.

The five-step case is the compatibility point: 218 has already been deleted, so the zero predecessor branch must pass through its existing 157 detour. Using the earlier three-step formula here would be wrong.

Lemma 6.1 (Final fold, arbitrary exterior)**.** *From state 0 on every common initial tape, R and $C = U_{209}$ halt equivalently. For a halting old run,*

$$T_C = T_R + 4n_0 + 2n_1, \qquad T_R \le T_C \le 5T_R,$$

where n_0 counts old read-zero visits to 216 and n_1 counts old read-zero visits to 180.

Proof. Inventory (8) establishes the tag invariant whenever either old state is entered. In the four exhaustive cases of Table 5, direct literal execution leaves both displayed cells unchanged, reads no other cell, and ends with the same head and the projected old target. No internal state is the halt label. Every remaining old row commutes with ρ, preserving its write and direction. Induction gives a positive block simulation. A new finite halt cannot occur inside a block, and each block consumes at least one new step, so the converse follows by the finite-horizon argument used for U210. Summing the four block lengths gives the clock identity. For arithmetic codes of finite-support tapes these are finite induction arguments formalizable in PA. □

The all-initial-tape assertion and time bound concern $R \to C$ alone. They are not extended to the preceding source-address transformations.

Relative to U210, U209 changes 15 retained rows and deletes both rows of 180. It changes 847 source addresses, including 508 changed by (4), while retaining the same 197 distinct syntactic subtrees.

7 Universal runtime and exact initialization

7.1 What the symbolic contracts quantify over

A canonical frame at native dispatch state 84 contains a left marker, a zero-padded source address, a header at distance $56 + g$, nine unary-coded primary registers, and a nonempty finite spare-register list, with zero exteriors. The physical register permutation is

$$(6, 2, 0, 4, 3, 7, 8, 5, 1).$$

A register value v is represented by the block $B(v) = 0\,1^{v+1}$. The symbolic inputs range over arbitrary natural values of all nine logical registers, an arbitrary natural extra gap g, an arbitrary first spare value, and an arbitrary remaining finite spare list.

For each source site, the verifier derives the exact primitive effect and successor frame. Conditional decrement is split into zero and positive cases, with the positive value written $v+1$. Increment appends the specified spare block, decrement and source arithmetic adjust the header/address representation as required, and branch operations leave the logical register store fixed. Nonhalt targets compare the complete oriented tape terms, head displacement, and dispatch state. Halt requires entry into native label 254.

The distinction between binary successor words and leaf addresses is checked: a target word may contain additional zero padding after its first decoded leaf, but it must contain no stray one bits there. An independent calculation confirmed agreement with ordinal source successors in all 5,083 nonhalt cases for each final candidate.

7.2 The proof rules used by the checker

The inherited engine is copied byte-for-byte from the Rotated package and its provenance is retained. It uses three execution rules:

1. A literal step applies a row of the actual table and updates the represented tape, state, motion, and clock.
2. A unary scan accelerates a literal self-loop in the scan direction across b^n, for arbitrary natural n. Induction on n justifies the resulting block and its exact duration.
3. A finite-list traversal uses a one-block theorem first rederived from the actual table with opaque outside sentinels. Each such theorem returns to the same state and direction, handles an arbitrary block value v, takes $v + 2$ steps, and preserves the exterior. Induction then traverses an arbitrary finite list.

The four one-block lemmas are regenerated on both final tables. Normalization uses literal word identities and the valid finite-list identity $C(L)1 = 1B(L)$, where $C(v) = 1\,0\,1^v$, together with its reversal. Its base case and one-block algebra are checked; list induction supplies the arbitrary-list form.

Clock and motion expressions use thirteen affine slots: a constant, the gap, nine primary values, the first spare value, and the total width of the remaining spare list. Every accepted primitive duration has nonnegative coefficients and strictly positive constant. These are symbolic induction certificates, not numerical tests at 5,084 sampled register settings. The engine is ordinary Python, so the report makes its small set of mathematical rules and their trust boundary explicit rather than calling it a proof-assistant kernel.

Fresh final-table verification	U210	U209
Guarded contracts	5,084	5,084
Symbolic inference applications	1,249,857	1,248,748
Literal rows covered	420/420	418/418
Minimum duration constant	5	5
Final internal-prefix checks	3,908	3,908
Maximum steps in those prefix checks	49	49
Semantic corruptions rejected	9	11

Table 6: Both complete runtime suites were freshly executed. Symbolic inference counts are certificate-replay counts, not a claim about the actual proof search's running time.

7.3 The exact blank startup

Both new tables have the same complete 48,140-transition blank prefix as the Research U211 parent. No changed row is visited. It ends at

$$q = 84, \qquad h = -2, \qquad \text{header} = 54, \qquad \text{source ordinal} = 136,$$

with source address `001100110101010`, logical register store

$$(1, 0, 13, 0, 0, 1, 0, 0, 1),$$

and 43 spare registers of value one. The checker compares the entire set of 120 nonzero tape cells, so the assertion includes the zero exterior, not just a finite viewing window.

The same ordinal/store is reached after 141 source primitives from ordinal 106 with initial store

$$(1, 0, 4, 0, 0, 1, 0, 0, 1).$$

This finite source bridge is replayed. The inherited startup-suffix proof supplies the further logical meaning: the incoming cache value 1 is used arithmetically and is not assumed to be a theorem; the suffix creates nine theorem-valued heads above opaque tail 4.

That tail would be unsafe if exposed. The retained scheduler argument bounds every command-prefix debt by two, leaving a permanent margin of seven explicit heads. A ghost explicit stack length handles the pairing alias $P(0, 0) = 0$; a numerical stack-length decoder is not assumed. The retained strong proof compiler works above arbitrary underlying stacks and produces cofinally many successful candidate codes. These arguments provide soundness and completeness after the actual startup and finite search cutoff. Neither is replaced by the finite 48,140-step test.

7.4 Independent checks and negative controls

The finite checkers do not import the construction builder. They derive all 3,908 internal source-prefix bindings from literal execution, use two complete CFG resolvers, test closure leaf by leaf, check full incoming inventories and every table row, and verify dense renaming. The U209 audit also checks the two arithmetic entry identities and four Research-selector cases on the final table.

The nine U210 and eleven U209 negative controls deliberately damage writes, moves, helper targets, endpoint rows, tag inventories, or the 171 repair. They are rejected by semantic checks rather than only by a hash mismatch. Both independent checkers produce byte-identical receipts normally and under `python3 -O`, because their obligations use explicit exceptions. The full symbolic engine uses assertions and rejects optimized Python.

8 Proof of the machine-to-consistency composition

Lemma 4.1 identifies all source executions by their ordinals and logical stores. The final runtime contracts show that every source primitive is simulated by a finite nonempty block of the corresponding literal machine. Their complete target frames maintain the induction invariant, including the gap and spare-register list. The unique halt primitive reaches the separate halt label; the other contracts return to dispatch with the prescribed successor and exclude earlier termination.

The exact startup places both final machines in the calibrated common interior source configuration. Consequently the source simulation applies to their actual blank runs. The inherited parent analysis gives the same source interpretation for the selected Research U211. This proves Theorem 1.1. It is not necessary to claim that every temporary reassociation table implements its temporary source.

The source proof search is unchanged, so its retained calibration gives

$$\mathrm{Halt}(U_{209}, \mathrm{blank}) \iff (\mathcal{D}_U \vdash 1) \iff (\mathrm{PA} \vdash \bot)$$

as the corresponding arithmetized equivalence. The source compiler, protected-tail proof, and finite-refutation translations discussed in Section 3 are the explicit inherited links. Negating the existence of a finite refutation yields Theorem 1.2.

Finite tables, bounded local paths, syntax transformations, and natural/list induction are all arithmetizable in PA. This explains the claimed formalizability. It does not turn a Python replay into a checked exported PA derivation. The archive contains no newly generated end-to-end proof object for U209 in Lean, Rocq, or MM0.

9 Reproduction, deliverables, and remaining work

The package is self-contained and uses Python 3.10 or later with its standard library. From its extracted root:

```
python3 reproduce.py --quick
python3 reproduce.py
python3 reproduce.py --both
```

The quick mode checks all frozen hashes, reconstructs both machines, binds the three comparison snapshots to the original archives, replays the two independent finite audits and both startups, and checks the delivered runtime records as data. It explicitly skips new universal derivations. The default additionally regenerates all U209 contracts. The `-both` mode regenerates both complete suites and requires exact equality with the delivered per-case symbolic records. Runtime depends on the computer; the fresh derivations here took a few minutes per table.

Generated replay outputs go under `build/`; delivered evidence is not overwritten. The complete payload is hash-indexed in `MANIFEST.json`. To rebuild this report, use a standard LaTeX installation:

```
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

Verify the delivered manifest before rebuilding the PDF, whose metadata can change with the TeX installation.

Location	Contents
`machine/`	Both dense and native tables and exact state bijections.
`source/`, `intermediates/`	Final source maps and immediate prefold tables.
`code/`	Deterministic builder, independent checkers, comparison, startup, and the unchanged inherited symbolic engine.
`evidence/`	Fresh complete runtime cases and summaries, independent finite receipts, startup frames, and source-stage diagnostics.
`comparison/`	Three exact input snapshots, pairwise census, and fresh original finite replay logs.
`inputs/`	All four original archives and the short construction recipes.
`proofs/`	Detailed theory, runtime, comparison, and new-reduction audit notes.

The research contribution established here is relative to the supplied proposals: previously separate savings have been made compatible through source reorganization that respects the exact context of a shared helper. No global minimality, independent record ratification, or priority claim is made. The natural next verification milestone is to export the local context certificates and source/runtime simulation to a small proof-assistant kernel, and then connect that development to the retained arithmetic calibration. The concrete machine and its current proof boundary are fully specified without that additional work.

A Complete dense transition table for U209

Start in state 0 on an all-zero tape. States $0, \ldots, 208$ are working states; 209 is the separate halt label. An entry wDr writes w, moves in direction D, and enters r. Each displayed row supplies both binary-symbol transitions. The equivalent plain-text table is `machine/u209.tm`.

q	$\delta(q,0)$	$\delta(q,1)$	q	$\delta(q,0)$	$\delta(q,1)$	q	$\delta(q,0)$	$\delta(q,1)$	q	$\delta(q,0)$	$\delta(q,1)$
0	1R92	1R0	53	0L40	1L53	106	0R61	1R42	159	0R19	1R171
1	0L1	1L2	54	0R22	1R54	107	0R121	1R8	160	0R164	1R187
2	0L3	1L2	55	0L11	0R50	108	0R101	1R122	161	0R159	1R96
3	0L5	1L2	56	0R60	0L16	109	0L208	0L95	162	0R125	1R164
4	1R6	0L4	57	0R69	1R70	110	0R124	1R104	163	0R97	1R162
5	0L9	1L2	58	0R61	1R71	111	0R115	1L72	164	0L190	1R142
6	0R6	0R10	59	0R54	1R59	112	0R165	1R181	165	0L66	1R173
7	0R190	1R159	60	0R49	0R25	113	0R160	1R184	166	0R146	1R172
8	0R14	1R13	61	0L28	1L72	114	0R148	1R146	167	0R199	1R201
9	0L15	1L2	62	0L117	1L30	115	0R11	1R129	168	0R161	1R143
10	0L16	0R10	63	0R73	1R63	116	0L131	0L1	169	0R148	1R87
11	0L26	1R128	64	0R97	1R150	117	0R39	1R72	170	0R161	1R96
12	0R19	1R20	65	1R185	1L53	118	0R61	1R132	171	0R181	1R122
13	0L89	1L31	66	0R144	1R145	119	0R61	1R133	172	0L193	1R60
14	0L11	0R22	67	0R76	1R209	120	0R70	0L16	173	0R145	1R88
15	0L23	1L2	68	0L197	1R31	121	0R14	1R132	174	0R152	1R202
16	1L1	0L1	69	0R41	1R62	122	0R21	1R134	175	0R140	1R84
17	0R24	0R32	70	0L89	1R78	123	0R151	1R189	176	0R175	1R75
18	0R98	1R113	71	0R55	1L79	124	0R137	1L20	177	0R126	1R169
19	0R28	1R29	72	0R80	1R81	125	0R130	1R180	178	0R138	1R116
20	0R30	1R31	73	1R0	1R63	126	0R204	1R200	179	0R97	1R51
21	0R24	0R59	74	0R74	1R82	127	0R75	1R176	180	0R168	1R51
22	0R33	1R22	75	0L145	1L141	128	0R75	1R206	181	0R17	1L131
23	0L34	1L2	76	0R85	1R86	129	0R203	1R157	182	0R143	1R149
24	0L35	0R0	77	0R20	1R38	130	0R12	1R141	183	0R76	1R116
25	0R36	1R25	78	0R89	1L132	131	0R30	1R69	184	0R106	1R124
26	0R37	1R38	79	0L108	1R56	132	0R39	1L124	185	1L40	1L109
27	0R39	0L40	80	0R49	0R33	133	0R153	0L16	186	0R175	1R200
28	0R41	1R42	81	0R91	0L42	134	0R154	0L16	187	0R182	1R183
29	0R43	1R44	82	0R178	1R77	135	0R88	1R136	188	0R66	1R205
30	0R45	1R46	83	0R96	1R12	136	0R197	1R140	189	0R140	1R139
31	0R21	1R47	84	0R88	1R97	137	0R24	0R48	190	0R84	1R191
32	0R48	1R32	85	0R207	1R21	138	0R99	1R37	191	0R157	1R186
33	0R32	1R33	86	0R116	1R100	139	0R172	1R148	192	0R64	1R7
34	0L74	1L2	87	0R174	1R167	140	0R96	1R159	193	0R150	1R127
35	1R35	1L1	88	0R102	1R103	141	0R175	1R166	194	0R18	1R94
36	0R50	1R36	89	0R80	1R79	142	0R12	1R159	195	0R150	1R173
37	0R17	1R111	90	0R101	1R196	143	0R88	1L166	196	0R104	1R96
38	0R13	1R52	91	0R105	1R106	144	0R88	1R158	197	0R90	1R88
39	0L11	0R36	92	0R65	0R0	145	0L152	1L163	198	0R200	1R156
40	0L4	1L53	93	0R27	1R107	146	0R84	1R88	199	0R96	1R123
41	0R24	0R25	94	0R183	0L208	147	0R159	1R12	200	0L195	1R195
42	0R13	0L16	95	1L109	1L95	148	0R83	1R143	201	0R202	1R198
43	0R55	1R56	96	0R19	1R110	149	0R144	1R155	202	0L123	1R168
44	0R57	1R58	97	0R60	1R60	150	0L66	1L141	203	0R197	1R28
45	0R24	0R54	98	0R68	1R135	151	0R150	1R170	204	0R163	1R193
46	0R60	1R61	99	0R93	1R108	152	0R159	1R114	205	0R146	1R177
47	0R62	0L16	100	0R105	0L208	153	0R97	1R97	206	0R161	1R194
48	0R25	1R48	101	0R117	1R118	154	0R49	0R32	207	0R45	1R100
49	0L35	0R63	102	0R71	1R58	155	0R192	1R188	208	0L4	0L208
50	0R16	1R50	103	0R57	1R119	156	0R150	1R112			
51	0R147	1R143	104	0R21	1R120	157	0L187	1R96			
52	0R67	1R68	105	0R21	0L10	158	0R168	1R179			

B Provenance and acknowledgments

The original U211 research packets and the unary-power-cover manuscript are credited to the supplied ProveIt research collection. Their original notices, nested dependency packets, and licenses are preserved unchanged in the retained archives. The runtime engine and source-language utilities are reused with their file-level provenance, not represented as newly authored proof infrastructure.

The construction inherits the proof-search architecture associated with the m254 reference development [6], the K215 verification lineage [4, 3], and the Theory K work [5]. Their respective fixed-machine theorems retain their original scope. New work in this package is the comparison, compatible U210 composition, shared-helper eight-factor reassociation, combined U209 fold, associated proofs, independent audits, and reproducible final artifact assembly.

References

[1] V. Reshetnikov, *ProveIt incoming research collection*, snapshot `da4ab9dd68ccfc47ac852118601eaf1fdae53795`. https://github.com/VladimirReshetnikov/ProveIt/tree/da4ab9dd68ccfc47ac852118601eaf1fdae53795/docs/incoming. The three exact U211 ZIP filenames and full hashes are in the package provenance.

[2] *Unary power-cover theory and arithmetic calibration*, `pa_unary_power_cover.zip`, in the same snapshot. https://github.com/VladimirReshetnikov/ProveIt/blob/da4ab9dd68ccfc47ac852118601eaf1fdae53795/docs/incoming/pa_unary_power_cover.zip. The original archive is included unchanged.

[3] E. Glazer, *K215 halts iff ZFC is inconsistent (Metamath Zero, PA)*. https://github.com/elliotglazer/k215-pa.

[4] V. Reshetnikov, *ProveIt K215 verification sources*. https://github.com/VladimirReshetnikov/ProveIt/tree/main/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215.

[5] E. Glazer, *K230 and Theory K*. https://github.com/elliotglazer/k230-rocq/blob/main/README.md.

[6] *Busy-beaver ZF/m254 reference development*. https://github.com/jeff87654/busy-beaver/tree/zf-254/ZF/m254.